



TaskFlow

Sistema de Gerenciamento de Tarefas
Relatório Técnico do Projeto

Trabalho: Trabalho 3 — Gestor de Tarefas "TaskFlow"

Stack: Next.js · TypeScript · Firebase · Tailwind CSS

Data: Junho de 2026

Sumário

1.	Visão Geral da Arquitetura	
2.	Estrutura de Pastas e Arquivos	
3.	Modelagem de Dados (Interfaces TypeScript)	
4.	Arquitetura do Sistema (Diagrama em Camadas)	
5.	Funcionalidades Implementadas	
6.	Desafios Técnicos e Soluções	
7.	Repositório GitHub e Guia de Instalação	
8.	Deploy na Vercel	
9.	Considerações Finais	

1. Visão Geral da Arquitetura

O TaskFlow é um sistema completo de gerenciamento de tarefas construído sobre uma arquitetura moderna de aplicações web, combinando a renderização híbrida do Next.js (App Router) com um backend serverless fornecido pelo Firebase. A escolha dessas tecnologias permite que o projeto seja escalável, rápido de desenvolver e fácil de manter, sem a necessidade de gerenciar infraestrutura de servidor própria.

1.1 Como as tecnologias se conectam

O **Next.js** atua como o núcleo da aplicação, responsável por roteamento, renderização de páginas e organização de componentes através do App Router. Toda a interface é construída com **TypeScript**, garantindo tipagem forte em modelos de dados como `Task`, `Subtask` e `UserProfile`, reduzindo erros em tempo de desenvolvimento e melhorando a manutenção do código a longo prazo.

A camada de dados e autenticação é delegada ao **Firebase**. O **Firebase Authentication** gerencia o ciclo de vida do usuário — cadastro, login por e-mail e senha (com verificação obrigatória), login social via Google e GitHub, exclusão de conta e proteção de sessão. Já o **Firestore**, banco de dados NoSQL orientado a documentos, armazena as coleções `users` e `tasks`, consultadas diretamente do lado do cliente através do SDK do Firebase, filtradas por `userId` para isolar os dados de cada usuário.

A estilização é feita com **Tailwind CSS**, utilizando um tema escuro consistente baseado em tons de slate e indigo. Para a camada visual de dados, o dashboard combina componentes inspirados no **Tremor** com o motor de renderização do **Recharts** para construir gráficos de distribuição de tarefas por status, prioridade e atividade recente. A interatividade de arrastar-e-soltar do quadro Kanban é implementada com **Dnd Kit**, enquanto a visualização de calendário utiliza o **FullCalendar**. As animações de entrada da landing page são feitas com **Framer Motion**, e os ícones em toda a aplicação vêm da biblioteca **Lucide React**.

Para validação de formulários, o projeto usa **React Hook Form** em conjunto com **Zod** através do `zodResolver`, garantindo validações consistentes tanto para os formulários de autenticação (cadastro e login) quanto para o formulário de tarefas (título, prioridade, status, data de vencimento e subtarefas).

1.2 Fluxo de dados resumido

Fluxo típico de uma operação:

Usuário interage com a UI (Client Component) → Hook customizado (ex: `useAuthActions`, `useTasksContext`) → Camada de serviço (`auth.service.ts`, `task.service.ts`) → SDK do Firebase (Auth / Firestore) → Resposta atualiza o estado global via Context API → Re-renderização automática de todos os componentes inscritos.

1.3 Gerenciamento de estado global

Duas instâncias de Context API orquestram o estado compartilhado da aplicação: o **AuthContext**, que escuta mudanças de autenticação via `onAuthStateChanged` e expõe o usuário atual para toda a árvore de componentes; e o **TasksContext**, que centraliza a lista de tarefas do usuário e suas operações de CRUD, garantindo que alterações feitas em uma página (por exemplo, mover uma tarefa no Kanban) sejam refletidas instantaneamente em outras páginas (Dashboard, Lista de Tarefas, Calendário) sem necessidade de recarregar dados.

2. Estrutura de Pastas e Arquivos

O projeto segue uma estrutura modular dentro do diretório `src/`, separando claramente páginas, componentes, lógica de negócio, hooks customizados, tipos e contextos globais.

```
src/
├── app/                                # Rotas (Next.js App Router)
│   ├── layout.tsx                     # Layout raiz (AuthProvider + TasksProvider)
│   ├── globals.css                   # Estilos globais e temas dark
│   ├── page.tsx                       # Redirecionamento raiz
│   ├── login/page.tsx                # Tela de login
│   ├── register/page.tsx             # Tela de cadastro
│   ├── dashboard/page.tsx            # Dashboard com métricas e gráficos
│   ├── tasks/page.tsx                # CRUD completo de tarefas
│   ├── kanban/page.tsx               # Quadro Kanban com Drag & Drop
│   └── calendar/page.tsx             # Calendário de vencimentos
├── components/
│   ├── auth/
│   │   ├── AuthCard.tsx              # Wrapper visual de autenticação
│   │   ├── InputField.tsx            # Input reutilizável com toggle de senha
│   │   ├── LoginForm.tsx             # Formulário de login completo
│   │   ├── RegisterForm.tsx          # Formulário de cadastro completo
│   │   ├── SocialButton.tsx          # Botão de login social
│   │   ├── PasswordStrengthBar.tsx   # Barra de força de senha
│   │   └── DeleteAccountModal.tsx    # Modal de exclusão de conta
│   ├── tasks/
│   │   ├── TaskCard.tsx              # Card de tarefa com progresso
│   │   ├── TaskForm.tsx              # Modal de criar/editar tarefa
│   │   ├── TaskFilters.tsx           # Filtros de busca/prioridade/status
│   │   └── SubtaskList.tsx           # Lista de subtarefas + barra de progresso
│   ├── dashboard/
│   │   ├── StatsCards.tsx            # Cards de KPI
│   │   ├── DashboardCharts.tsx       # Gráficos (donut, barras, área)
│   │   ├── RecentTasks.tsx           # Lista das tarefas recentes
│   │   └── OverdueTasks.tsx          # Painel de tarefas vencidas
│   ├── kanban/
│   │   ├── KanbanColumn.tsx          # Coluna droppável
│   │   ├── KanbanCard.tsx            # Card arrastável
│   │   └── KanbanTaskModal.tsx       # Modal de detalhes da tarefa
│   ├── calendar/
│   │   ├── CalendarView.tsx          # Wrapper do FullCalendar
│   │   └── CalendarTaskModal.tsx     # Modal de detalhes ao clicar
│   └── landing/
│       ├── Navbar.tsx
│       ├── Hero.tsx
│       ├── Features.tsx
│       ├── DemoMockup.tsx
│       ├── CTA.tsx
│       └── Footer.tsx
├── services/                           # Regras de negócio / acesso a dados
│   ├── auth.service.ts               # Login, cadastro, logout, exclusão de conta
│   └── task.service.ts               # CRUD de tarefas no Firestore
├── lib/
│   ├── firebase.ts                   # Inicialização do Firebase SDK
│   └── validations.ts                # Schemas Zod (auth + tarefas)
├── hooks/
│   ├── useAuthActions.ts             # Ações de autenticação + navegação
│   └── useTasks.ts                   # Hook local de tarefas (legado)
└── types/
```

```
| |— user.ts                # UserProfile, AuthState, formulários
| |— task.ts               # Task, Subtask, Priority, Status
|
|— contexts/
| |— AuthContext.tsx       # Estado global de autenticação
| |— TasksContext.tsx     # Estado global de tarefas
|
|— middleware.ts           # Proteção de rotas autenticadas
```

3. Modelagem de Dados (Interfaces TypeScript)

A tipagem forte do projeto garante que os dados trafegados entre componentes, hooks e serviços sigam sempre a mesma estrutura. As interfaces principais são `Task` e `Subtask`, complementadas por tipos auxiliares de prioridade e status.

3.1 Interface Task e Subtask

```
export type Priority = "baixa" | "media" | "alta";
export type Status = "pendente" | "em_progresso" | "concluida";

export interface Subtask {
  id: string;
  title: string;
  completed: boolean;
}

export interface Task {
  id: string;
  userId: string;
  title: string;
  description: string;
  priority: Priority;
  status: Status;
  dueDate: string; // formato ISO: YYYY-MM-DD
  subtasks: Subtask[];
  createdAt: Date;
  updatedAt: Date;
}

export interface TaskFormData {
  title: string;
  description: string;
  priority: Priority;
  status: Status;
  dueDate: string;
  subtasks: Subtask[];
}
```

3.2 Interface UserProfile

```
export interface UserProfile {
  uid: string;
  name: string;
  email: string;
  photoURL?: string;
  emailVerified: boolean;
  provider: "email" | "google" | "github";
  createdAt: Date;
  updatedAt: Date;
}
```

3.3 Validação com Zod

Cada interface possui um schema Zod correspondente, usado pelo `react-hook-form` através do `zodResolver` para validar os formulários antes do envio:

```
export const taskSchema = z.object({
  title: z.string().min(1, "Título é obrigatório.").max(100),
  description: z.string().max(500).optional().default(""),
  priority: z.enum(["baixa", "media", "alta"]),
  status: z.enum(["pendente", "em_progresso", "concluida"]),
  dueDate: z.string().min(1, "Data de vencimento é obrigatória."),
});
```

4. Arquitetura do Sistema (Diagrama em Camadas)

O sistema segue uma arquitetura em camadas que separa claramente apresentação, lógica de aplicação, acesso a dados e infraestrutura externa:

🏠 Camada de Apresentação (UI)

Páginas (App Router) · Componentes React · Tailwind CSS · Framer Motion



⚙️ Camada de Estado e Lógica de Aplicação

Context API (AuthContext, TasksContext) · Hooks customizados (useAuthActions) · React Hook Form + Zod



🔗 Camada de Serviços (Regras de Negócio)

auth.service.ts · task.service.ts — funções puras que encapsulam chamadas ao Firebase



🔥 Camada de Infraestrutura (Firebase)

Firebase Authentication · Firestore Database · Hospedagem Vercel

Por que essa separação importa: a camada de serviços isola toda a comunicação com o Firebase, permitindo que componentes de UI nunca chamem o SDK diretamente. Isso facilita testes, manutenção e uma futura migração de backend, caso necessário, sem reescrever a interface.

5. Funcionalidades Implementadas

Módulo	Status	Observações
Cadastro de usuário	Completo	Validação Zod, senha forte, confirmação de senha
Login (e-mail, Google, GitHub)	Completo	Verificação obrigatória de e-mail, proteção de rotas
Exclusão de conta	Completo	Re-autenticação obrigatória antes da exclusão
CRUD de tarefas	Completo	Criar, listar, editar, excluir com filtros
Subtarefas e progresso	Completo	Barra de progresso calculada automaticamente
Dashboard com gráficos	Completo	3 gráficos: donut, barras e área temporal
Quadro Kanban	Completo	Drag and drop com Dnd Kit entre 3 colunas
Calendário	Completo	FullCalendar com modal de detalhes ao clicar
Landing page	Completo	Navbar responsiva, hero, features, mockup, CTA, footer

5.1 Tecnologias utilizadas por módulo

Categoria	Tecnologia
Framework	Next.js 14 (App Router)
Linguagem	TypeScript
Backend / Auth	Firebase Authentication + Firestore
Estilização	Tailwind CSS
Formulários	React Hook Form + Zod
Gráficos	Recharts (com componentização Tremor)
Drag & Drop	Dnd Kit
Calendário	FullCalendar
Animações	Framer Motion
Ícones	Lucide React
Notificações	Sonner

6. Desafios Técnicos e Soluções

Durante o desenvolvimento, três desafios técnicos se destacaram pela complexidade de diagnóstico e pela importância da solução adotada.

Desafio 1 — Sincronização de estado entre páginas no Kanban

Inicialmente, cada página (Dashboard, Tarefas, Kanban) possuía sua própria instância do hook `useTasks`, fazendo com que uma mudança de status no quadro Kanban não fosse refletida no Dashboard sem recarregar a página.

Solução adotada: migração para um `TasksContext` global usando Context API do React, instanciado uma única vez no layout raiz da aplicação. Todas as páginas passaram a consumir o mesmo estado compartilhado através do hook `useTasksContext()`, garantindo que qualquer alteração (criar, editar, mudar status, excluir) seja refletida instantaneamente em todas as telas.

Desafio 2 — Tema escuro em bibliotecas de terceiros (Tremor / Recharts)

Bibliotecas de gráficos como o Tremor utilizam variáveis CSS próprias (ex: `--tremor-content`) e classes condicionais de modo escuro (`dark:text-...`) que não são ativadas automaticamente apenas pela presença de fundo escuro na aplicação — é necessário que o Tailwind seja configurado com `darkMode: "class"` e que a classe `dark` esteja presente no elemento `<html>`, além de variáveis CSS customizadas declaradas manualmente no `globals.css`.

Solução adotada: configuração explícita de `darkMode: "class"` no `tailwind.config.ts`, adição de `className="dark"` na tag `<html>` do layout raiz, e declaração manual das variáveis `--tremor-content`, `--tremor-background` e equivalentes dentro do seletor `.dark` no CSS global, garantindo legibilidade de textos, eixos e legendas em todos os gráficos.

Desafio 3 — Otimização do Drag and Drop no Kanban com muitas tarefas

Ao implementar a movimentação de cards entre colunas com Dnd Kit, o desafio foi garantir uma experiência fluida mesmo quando uma coluna concentra muitas tarefas, evitando travamentos durante o arraste e inconsistências entre o estado visual e o estado persistido no Firestore.

Solução adotada: uso de atualização otimista do estado local (`localTasks`) durante o evento `onDragOver`, fazendo a UI responder instantaneamente ao movimento do mouse, enquanto a persistência real no Firestore ocorre apenas no evento `onDragEnd`. Em caso de falha na escrita, o estado é revertido para os dados originais (`allTasks`), evitando inconsistência entre o que o usuário vê e o que está salvo.

7. Repositório GitHub e Guia de Instalação

7.1 Estrutura recomendada do README.md

O repositório do projeto deve conter um arquivo `README.md` na raiz com o seguinte roteiro, servindo como guia passo a passo para qualquer pessoa clonar e executar o projeto localmente:

Passo 1 — Clonar o repositório

```
git clone https://github.com/seu-usuario/taskflow.git
cd taskflow
```

Passo 2 — Instalar as dependências

```
npm install
```

Passo 3 — Configurar variáveis de ambiente

Criar um arquivo `.env.local` na raiz do projeto com as credenciais do Firebase:

```
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
```

Passo 4 — Configurar o Firebase Console

- Criar um projeto no [Firebase Console](#)
- Ativar Authentication com os provedores Email/Senha, Google e GitHub
- Ativar o Firestore Database em modo de produção ou teste
- Adicionar `localhost` e o domínio de produção em "Authorized domains"

Passo 5 — Executar localmente

```
npm run dev
```

A aplicação estará disponível em `http://localhost:3000`

7.2 Scripts disponíveis

Comando	Descrição
<code>npm run dev</code>	Inicia o servidor de desenvolvimento
<code>npm run build</code>	Gera a build de produção
<code>npm run start</code>	Executa a build de produção localmente
<code>npm run lint</code>	Executa o linter do projeto

8. Deploy na Vercel

8.1 Passos para publicar

- 1 Conectar o repositório GitHub à conta Vercel através do botão "Add New Project".
- 2 Configurar as variáveis de ambiente do Firebase em *Settings* → *Environment Variables*, replicando exatamente o conteúdo do `.env.local`.
- 3 Adicionar o domínio gerado pela Vercel (ex: `taskflow.vercel.app`) na lista de *Authorized domains* do Firebase Authentication.
- 4 Atualizar a Callback URL do OAuth App do GitHub para incluir a URL de produção do Firebase (`https://seu-projeto.firebaseio.com/__/auth/handler`).
- 5 Disparar o deploy e validar login, cadastro e operações de CRUD em produção.

Atenção: erros de login social em produção quase sempre estão relacionados a domínios não autorizados no Firebase ou variáveis de ambiente não configuradas na Vercel — esses dois pontos devem ser os primeiros a verificar em caso de falha.

9. Considerações Finais

O desenvolvimento do TaskFlow permitiu a aplicação prática de conceitos centrais do desenvolvimento web moderno: arquitetura em camadas, tipagem forte com TypeScript, gerenciamento de estado global com Context API, integração com serviços de backend-as-a-service (Firebase) e construção de interfaces ricas e interativas (drag-and-drop, gráficos dinâmicos, calendário).

Os principais aprendizados do projeto envolveram a importância de uma camada de serviços bem definida para isolar a lógica de acesso a dados da interface, a necessidade de testar cuidadosamente a integração de bibliotecas de terceiros com sistemas de temas customizados, e o valor de atualizações otimistas de UI para manter a fluidez em interações como o arrastar-e-soltar do Kanban, mesmo com a latência inerente de operações assíncronas contra o Firestore.